

Summary of work completed

Moved away from the class diagram to implementing the classes within header and cpp files. Started with focusing on the core gameplay classes such as player, game flow, map, locations, inventory, items, NPCs, and input/display systems. From there we took turns to change the design of the classes to ensure they worked how we wanted. Implemented the design changes we realized needed to be done. Then refined how each class interacts with each other to create the game loop in main.

Design changes and why

Food was changed to be a public getter to let other classes use the value without exposing the variable. The location generalization was removed because it was unnecessary as we wanted the design to be simpler. The player gained an extra helper and getter method and the move was adjusted into moveaction. These changes were made because it was easier for other classes to check players stat, inventory and location during the game loop. Inventory gained helper methods like access by index and size and its "has" behavior was expanded to be more complex. This was done so inventory contents could be displayed easier in the game loop. WorldMap gained helper methods like getting a location index and checking current position. We added these because the game needed a way to support movement and map. Location got a couple getters and helpers and canExit was changed to work with the map. This made it easier for movement logic to work. In Game we added a constructor and a helper called isOver(). These were added to make everything cleaner and let us check game state. UserInput we added extra input helpers like limiting choices and handling with yes or no answers. We did this so input validation could be refused and kept game logic. In ScreenDisplay we adjusted map drawing so it used the players current location and its choices display was more specific to better match what the game needed to show. NPC was simplified by removing the shared locations field and base interact so that the NPC behavior could be in subclasses instead of one interaction function. HelpNPC only changes some naming to clean up stuff to better match how we were using it. ShowNPC was simplified by removing the price structure because we realized the shop was more complex than it needed to be. CraftingRecipe changed its output from Item to Item* so that recipes could make different item subclasses. Item gained a getter so that it could easily tell what kind of item it was working with during the game loop. Weapon was reduced to keep the class more minimal until we figured out combat stuff later. Lastly, ItemCollection was added to help organize and manage items in the project. Overall, most of our changes were either adding or taking away getters or helpers in the classes to change how the designs function to be simpler.

Reflection on team workflow and communication

As a team we kept good communication throughout the timeframe to ensure equal work was first distributed in creating all the classes. We met a couple times in person to discuss what we wanted to do. From there we started taking turns on general fixes needing to be done. Changing design, fixing errors, etc. This led to us all having an active hand on creating the project and then getting it to run successfully.

Lessons learned

Be open to changing the design when needed. Fight general frustration to get things working.
Learned more in depth the relationships between classes in handling access and pointers.
Learned more in depth how getters and helpers work to ensure design idea is followed to create
our game how we envisioned it